



Technical report · Group 04

Floe

A CDC lakehouse on Apache Iceberg, Spark and Trino

Streaming row-level changes from PostgreSQL through Debezium and Kafka into Apache Iceberg, optimised by Airflow and queried with Trino – built, measured and verified end to end.

Members	Aliyan Zulfiqar (Leader), Sehrosh Riaz
Assigned project	Apache Iceberg + Apache Spark
Instructor	Dr. Mohsin Raza Jafri
Lab engineer	Engr. Muhammad Abid Hussain
Repository	floe



Abstract

Operational databases are a poor place to run analytics, while classic nightly ETL copies whole tables and delivers stale data. We built a lakehouse that continuously replicates row-level changes from PostgreSQL into Apache Iceberg tables and serves them to SQL analytics with Trino. Debezium reads the PostgreSQL write-ahead log and publishes change events to Kafka; a Spark Structured Streaming job applies each micro-batch to Iceberg with an idempotent `MERGE`, evolving table schemas automatically. Tables live in S3-compatible object storage (MinIO) behind an Iceberg REST catalog. Airflow orchestrates Spark maintenance jobs – partition evolution, sorted compaction, delete-file rewriting, snapshot expiry and a GDPR-style purge – and Prometheus/Grafana monitor freshness and file health. On TPC-H scale factor 1 (1,650,030 rows) the pipeline delivered the full load to Iceberg at 4,844 rows/s end-to-end with checksum-verified correctness, and made individual changes queryable with a median end-to-end freshness of 7.3 s. Re-laying out the streamed tables removed all 105 merge-on-read delete files, cut the number of objects in storage by 89 % and made the Trino query suite 3.25× faster overall (up to 8.8× for a single query).

Keywords: lakehouse, change data capture, Apache Iceberg, Spark Structured Streaming, Debezium, Trino, compaction.

01 Introduction and motivation

Businesses want analytics over the same data their applications are changing right now: orders, balances, inventory. Running heavy analytical SQL directly on the operational database competes with transactions, and the traditional alternative – exporting full tables to a warehouse every night – is both expensive (every run copies everything) and stale (data is up to a day old). A *lakehouse* combines the low-cost, open-format storage of a data lake with the transactional table semantics of a warehouse; *change data capture* (CDC) feeds it incrementally by reading only the changes from the database log.

The project goal was to implement such a pipeline end-to-end with open-source components and to quantify its behaviour. We call the resulting system **Floe** – like an ice floe, a slab of Iceberg that keeps drifting with the flow of changes. The concrete objectives were:

1. Stream inserts, updates and deletes from PostgreSQL into Apache Iceberg tables with Debezium, Kafka and Spark.
2. Keep the lakehouse **correct** (identical to the source) and **fresh** (seconds behind), including when the source schema changes.
3. Demonstrate Iceberg's schema evolution, time travel and row-level deletes (including GDPR-style erasure).
4. Counter the small-file problem that streaming creates with compaction and partition optimisation jobs orchestrated by Airflow, and measure query performance and storage before and after.
5. Make the system observable (Prometheus/Grafana), reproducible (Docker Compose, scripts) and tested (unit, integration, CI).

02 Background

Change data capture. PostgreSQL records every committed change in its write-ahead log (WAL). With `wal_level=logical` a *logical replication slot* decodes the WAL into row changes; Debezium consumes the slot through the built-in `pgoutput` plugin, first snapshots existing rows and then emits one event per inserted, updated or deleted row, carrying the new row image, the operation, the commit time and the log sequence number (LSN). Because the slot retains WAL until the consumer confirms it, no change is lost while consumers are down.

Apache Iceberg is an open table format: a table is a tree of immutable files – Parquet data files, delete files, manifests and a metadata JSON file – and a catalog stores an atomic pointer to the current metadata file. Every commit produces a new *snapshot*, which gives ACID transactions with optimistic concurrency, time travel, schema evolution by column ID, and *hidden partitioning* whose specification can be changed without rewriting old data. Format v2 adds row-level deletes: with *merge-on-read*, an update writes the new row plus a small *position-delete* file instead of rewriting the data file (copy-on-write).

Engines. Spark Structured Streaming processes Kafka in micro-batches with checkpointed offsets; Trino is a distributed SQL engine that reads Iceberg tables directly from object storage.

03 System architecture

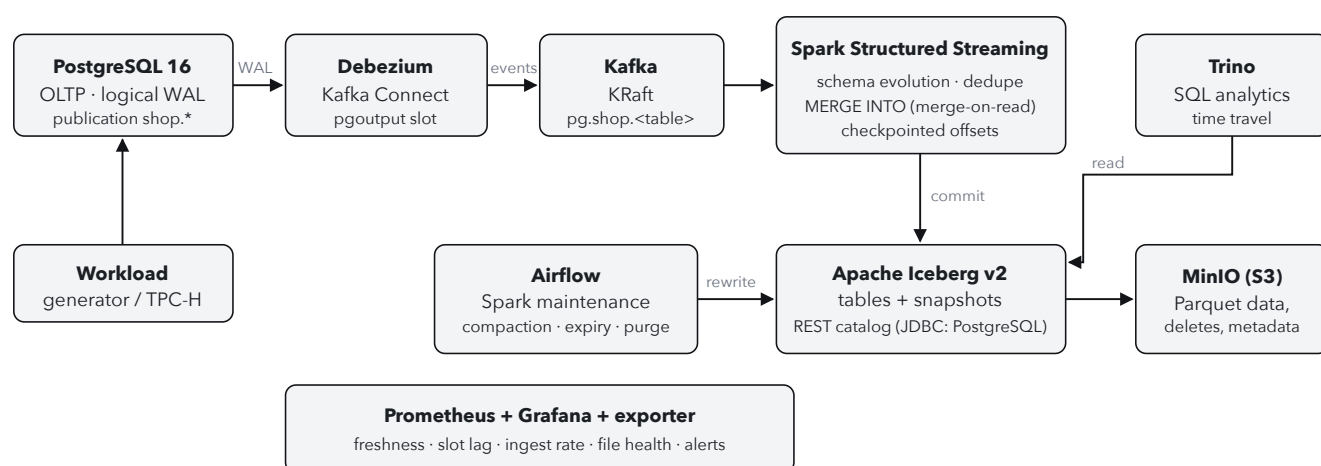


Figure 1 · Components and data flow. All services run as containers from one `docker-compose.yml`.

Figure 1 shows the pipeline. Every component is open source and runs as a container; the same design maps onto managed cloud services (Section 7). Table 1 lists the components and the reasons for each choice.

Layer	Component	Design choice and reasoning
Source	PostgreSQL 16	<code>wal_level=logical</code> ; publication <code>FOR TABLES IN SCHEMA shop</code> also covers tables created later.
CDC	Debezium 2.7 (Kafka Connect)	Snapshot + streaming in one connector; decimals sent as exact strings with source precision/scale; secrets from environment variables.
Transport	Kafka 3.9 (KRaft)	Durable, replayable log, one topic per table, ordered per key; zstd compression for the schema-carrying JSON events.
Processing	Spark 3.5 Structured Streaming	<code>foreachBatch</code> gives full SQL (MERGE) per micro-batch; checkpointed offsets.
Table format	Iceberg 1.9, format v2	Engine-neutral; merge-on-read keeps streaming writes cheap; partition evolution without rewrites.
Catalog	Iceberg REST catalog	Standard API used by both Spark and Trino; metadata pointers stored in PostgreSQL (concurrency-safe).
Storage	MinIO	S3 API locally; swaps for Azure Data Lake Storage / Amazon S3 in the cloud.
Query	Trino 476	Interactive SQL on Iceberg incl. metadata tables and time travel; also reads PostgreSQL for validation.
Orchestration	Airflow 2.10	Scheduling, retries and run history for Spark maintenance jobs.
Monitoring	Prometheus 3, Grafana 12	Custom exporter for lakehouse-specific signals plus Spark and MinIO metrics; alert rules.

Table 1 · Components.

04 Implementation

4.1 Change capture

An initialisation script creates three database roles (application, Debezium with `REPLICATION`, catalog) and the TPC-H shaped schema `shop` (region, nation, customer, orders) plus a heartbeat table. The connector configuration is applied idempotently by a one-shot container (`PUT /connectors/postgres-cdc/config`) that waits until the task is `RUNNING`. Key settings: `plugin.name=pgoutput`, `decimal.handling.mode=string` together with `column.propagate.source.type` (so `numeric(12,2)` arrives exactly and with its precision), heartbeats every 10 s, and `max_slot_wal_keep_size=4GB` on PostgreSQL so a stopped consumer cannot fill the source disk.

4.2 Streaming ingestion into Iceberg

The Spark job subscribes to `pg\shop\.*` (new tables are discovered automatically) and handles each micro-batch per table in five steps:

1. **Schema discovery.** Each event embeds its schema. The job collects the distinct schemas present in the batch (cutting the message text at the `"payload"` key, which is much cheaper than parsing every 3 KB event) and merges them, newest first. Debezium logical types are mapped to Spark/Iceberg types (dates, microsecond timestamps, zoned timestamps, decimals from source precision, binary).
2. **Table creation / schema evolution.** Unknown tables are created (format v2, merge-on-read, zstd; primary key recorded as a property). New source columns become `ALTER TABLE ... ADD COLUMN`; type changes Iceberg can promote in place (int→bigint, float→double, wider decimal) become `ALTER COLUMN ... TYPE`; anything else is cast to the existing type and logged.
3. **Deduplication.** Only the latest event per primary key (highest Kafka offset) is kept.
4. **MERGE.** One statement applies the batch:

```

MERGE INTO lakehouse.shop.orders t USING batch s ON t.o_orderkey = s.o_orderkey
WHEN MATCHED AND s._cdc_op = 'd' THEN DELETE
WHEN MATCHED THEN UPDATE SET ...           -- all source columns + CDC metadata
WHEN NOT MATCHED AND s._cdc_op <> 'd' THEN INSERT ...

```

Because the MERGE is idempotent, replaying a micro-batch after a crash (Spark re-runs the last uncommitted batch) yields the same table state; with checkpointed offsets this gives effectively-once results. Commits that collide with a concurrent maintenance job are retried with exponential backoff. Each row carries `_cdc_op`, `_cdc_source_ts` (commit time in PostgreSQL), `_cdc_lsn` and `_ingested_at`, which enable freshness measurement and change audits. Finally, per-table batch statistics are appended to `lakehouse.ops.ingest_batches`.

4.3 Maintenance and optimisation

A single Spark job (`maintenance.py`) exposes the Iceberg procedures as composable steps, and three Airflow DAGs schedule them: hourly housekeeping, a manual *layout optimisation*, and a *GDPR purge*. The layout optimisation evolves `orders` to `PARTITIONED BY months(o_orderdate)` with sort order `(o_orderdate, o_custkey)` and sorts `customer` by `(c_nationkey, c_custkey)`, then runs `rewrite_data_files` (sort strategy, 128 MB target, all deletes applied, partial progress so commits stay small), `rewrite_position_delete_files`, `rewrite_manifests`, `expire_snapshots` and `remove_orphan_files`. Partition evolution changes only metadata: the streaming job keeps running and immediately writes new data in the new layout. Orphan removal is fed with an S3 listing through `file_list_view` and is restricted by Iceberg to files older than 24 h, so files of in-flight commits are never deleted.

4.4 Monitoring, testing and reproducibility

A custom Prometheus exporter writes a heartbeat row into PostgreSQL every 15 s and reports how old the newest heartbeat visible through Trino is – the true end-to-end freshness – together with replication-slot lag, connector state, ingest rate, MERGE duration and per-table data/delete files, bytes and snapshots. Prometheus also scrapes Spark's built-in streaming metrics and MinIO. Alert rules fire on stale data, a stopped connector, WAL growth, small-file accumulation and a stalled stream. Correctness is checked by comparing row counts and an order-independent `checksum()` over all columns between the `postgresql` and `lakehouse` catalogs in Trino. The repository contains unit tests (type mapping, evolution rules, configuration), a full-stack integration test (load → workload → update/delete → schema change → maintenance, each followed by a consistency check), a GitHub Actions workflow, and scripts for setup, the scripted demo and clean-up.

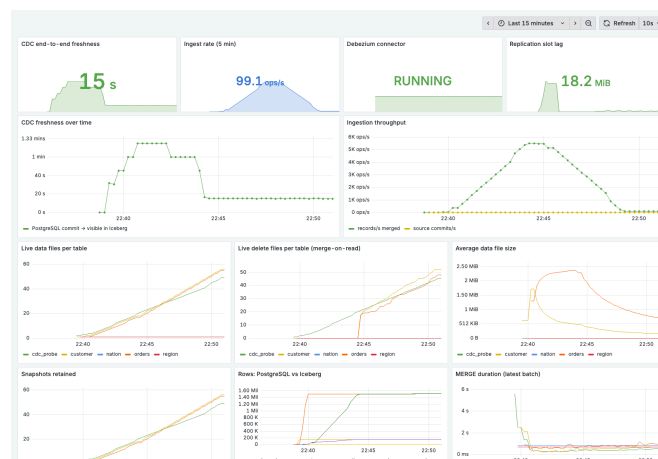


Figure 2 · Grafana dashboard during the final run: initial-load backlog, then steady freshness under the live workload while data and delete files accumulate.

05 Evaluation and results

RUN INFORMATION

Host	Apple silicon, 8 cores
Docker memory	8 GB
Spark	local mode, 10 s trigger
Batch limit	100 000 records
Dataset	TPC-H sf1 (Trino tpch)
Workload	100 ops/s, 5 min
Query timing	median of 5 + warm-up

QUALITY CONTROL - POSTGRESQL VS ICEBERG

customer · 149,932 rows	✓ PASS	nation · 25 rows	✓ PASS
orders · 1,513,192 rows	✓ PASS	region · 5 rows	✓ PASS
Initial load (checksums)	✓ PASS	Freshness probes visible	✓ PASS

All services ran on one laptop with local-mode Spark, so the numbers describe a single node rather than a cluster. The data is TPC-H scale factor 1 (150 000 customers, 1 500 000 orders) plus a synthetic OLTP workload of 45 % new orders, 20 % order updates, 10 % order deletes, 15 % balance updates and 5 % each new and deleted customers.

Query suite speed-up

3.25x

Eight Trino queries, median of 5 runs, before vs after the Airflow layout-optimisation DAG (up to 8.8x for a single query).

Headline results

4,844

rows/s end-to-end ingestion, 1,650,030 rows

7.3 s

median CDC freshness (p95 11.7 s)

−89 %

objects in storage (2,151 → 246)

0

checksum differences, PostgreSQL vs Iceberg

5.1 Ingestion throughput

Loading 1,650,030 TPC-H rows into PostgreSQL took 27 s; all of them were queryable in Iceberg, with matching checksums, 341 s after the load started – **4,844 rows/s end-to-end** through Debezium, Kafka, Spark and the MERGE commit. Across all 62 micro-batches (1,688,434 change events) the streaming job processed 3,575 events/s while busy (peak batch 7,752 events/s); median batch duration was 3.7 s (p95 20.5 s). The MERGE into orders itself took 0.9 s at the median (1.9 s at p95), even for 90 000-row batches; most of the batch time is JSON decoding of the self-describing Debezium events on a single local-mode executor, which is the first thing a cluster deployment (more executors) or Avro + Schema Registry would improve.

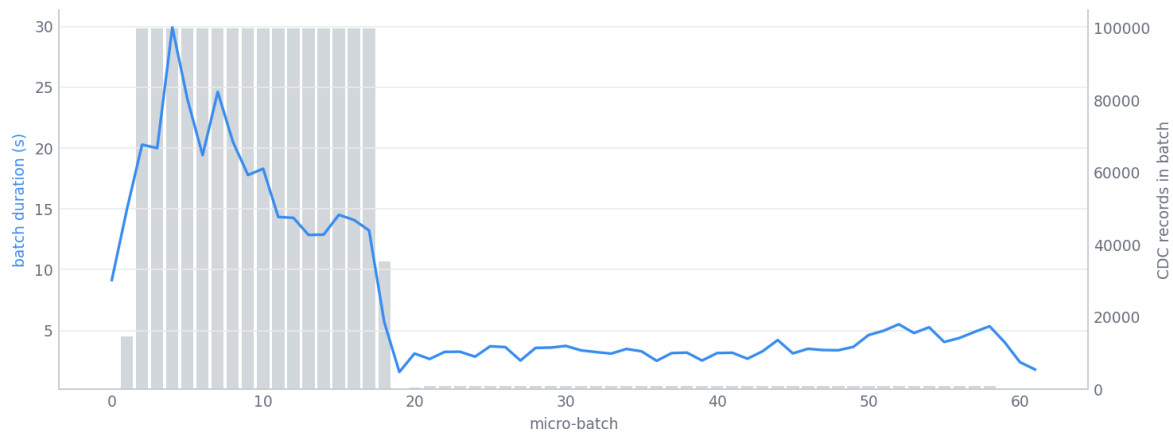


Figure 3 · Change events and processing time per micro-batch (initial load followed by the live workload).

5.2 CDC end-to-end freshness

While the workload was running, probe rows were inserted into PostgreSQL every 2 s and polled through Trino. Of 40 probes (0 timed out), the time from PostgreSQL commit to visibility in a Trino query was **7.3 s median**, 11.7 s at p95 and 12.7 s worst case (minimum 2.9 s). The pipeline-internal part (commit → MERGE start) had a median of 6.1 s. Freshness is bounded below by the 10 s trigger interval plus the MERGE and commit time; a shorter trigger lowers latency at the cost of more, smaller files – exactly the trade-off the maintenance jobs address.

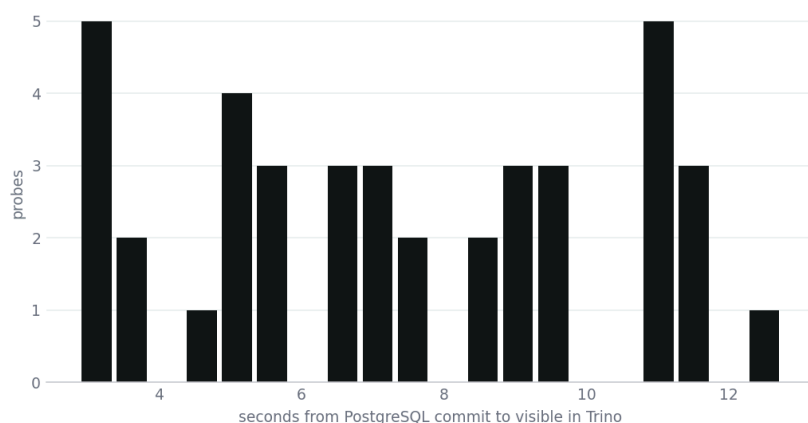


Figure 4 · Distribution of end-to-end freshness.

5.3 Small files, storage and query performance

Streaming writes leave the tables fragmented: after the load and workload the orders table consisted of 57 data files averaging 717 KB, plus 49 position-delete files that every query must merge at read time. The optimisation DAG rewrote them into 80 sorted files (average 0.6 MB) in monthly partitions and expired the superseded snapshots.

Table	Data files	Delete files	Avg data file	Snapshots
orders	57 → 80	49 → 0	717 KB → 581 KB	57 → 1
customer	58 → 1	56 → 0	157 KB → 8,630 KB	58 → 1

Table 2 · Live files before → after optimisation.

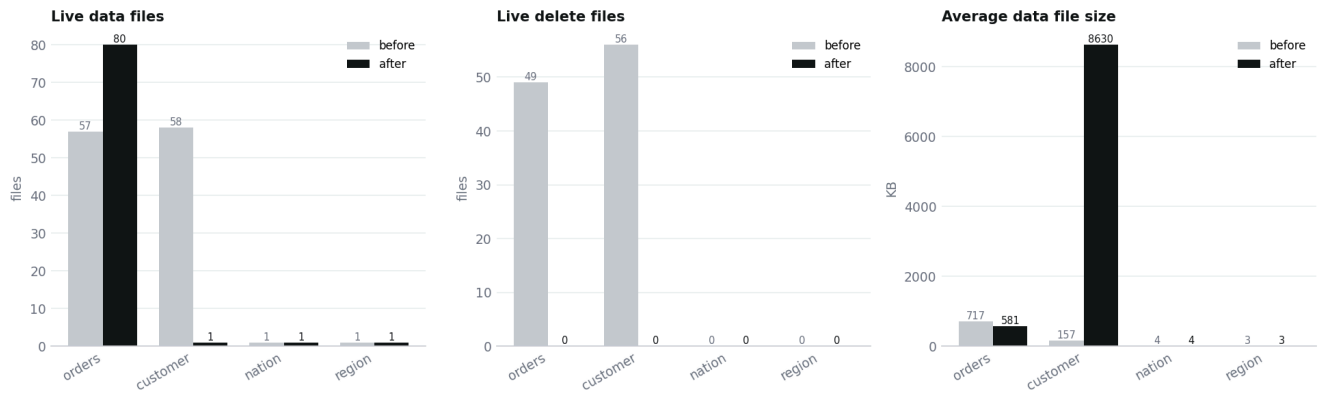


Figure 5 · Live data files, delete files and average file size per table.

Table	Before	After	Bytes change
orders	44.8 MB (1,141 obj.)	47.6 MB (163 obj.)	+6.2 %
customer	13.6 MB (1,000 obj.)	10.6 MB (67 obj.)	-22.4 %
nation	19.2 KB (5 obj.)	29.4 KB (8 obj.)	+53.4 %
region	17.8 KB (5 obj.)	27.9 KB (8 obj.)	+56.3 %

Table 3 · Object storage footprint in MinIO (all objects: data, deletes, manifests, metadata, old snapshots).

The number of objects in the warehouse fell by 89 % (2,151 → 246): 105 live delete files, the superseded small data files, their manifests and the metadata of expired snapshots were removed. Total bytes changed by only -0.4 % (58.5 MB → 58.3 MB), because five minutes of workload produce small delete files and the expired snapshots referenced little data that was not still live; the monthly layout also stores 80 separately compressed files. Byte savings grow with longer retention of update-heavy history, whereas the object count – which drives listing, planning and request costs on cloud object storage – is what streaming ingestion inflates and what compaction controls.

Table 4 compares the Trino query suite. Selective date queries benefit from partition pruning (whole months are skipped without reading files), and every query benefits from reading fewer files with no delete files to merge at read time. The point and customer lookups read *more* rows afterwards: streaming had written orders roughly in key order, which gave the original files narrow key ranges, while the new layout is sorted by date; they still became faster because far fewer files and no delete files are involved. Overall the suite ran **3.25× faster**; the largest gain was for *point lookup* (8.8×). The exception is *full aggregate* (0.95×): a scan of the whole table gains nothing from pruning and now opens 80 monthly files instead of 57 streaming files, which offsets the removed delete files – the granularity trade-off examined below.

#	Query	Before (ms)	After (ms)	Speed-up	Rows read before	Rows read after
Q01	point lookup	971	110	8.81×	1,231,343	1,513,192
Q02	single month	340	46	7.38×	1,513,192	19,488
Q03	quarterly revenue	1,287	174	7.39×	1,513,192	690,957
Q04	revenue by nation	8,089	1,381	5.86×	1,813,116	530,608
Q05	top customers	8,934	3,646	2.45×	1,813,056	1,813,056
Q06	full aggregate	950	1,004	0.95×	1,513,192	1,513,192
Q07	segment priority	750	178	4.22×	1,685,916	229,722
Q08	customer history	240	91	2.64×	1,493,627	1,513,192
total		21,561	6,630	3.25×		

Table 4 · Trino query latency (median of 5 runs) and rows read.

SPEED-UP PER QUERY

point lookup	8.81x	single month	7.38x
quarterly revenue	7.39x	revenue by nation	5.86x
top customers	2.45x	full aggregate	0.95x
segment priority	4.22x	customer history	2.64x

ABLATION · PARTITION GRANULARITY

We also compared a single fully compacted orders file with the same data in monthly partitions (80 files of ~570 KB each). Partitioning cut the rows read by the one-month query from 1,503,248 to 19,344 (latency -18 %; quarter query -20 %), but full-table queries became slower (+21 % and +35 %) because Trino has to open many small files. At TPC-H sf1 a month holds only ~20 000 orders; partitions should be sized to tens or hundreds of MB, so monthly partitioning pays off as volume grows, while at small scale a coarser transform (e.g. **years**) or sort order alone is preferable. Iceberg's partition evolution makes this a metadata-only decision that can be revisited.

5.4 Compaction overhead

The complete optimisation DAG took **170 s** of wall time, running as Spark local-mode jobs inside the Airflow container while the streaming job continued. Its cost is paid once per layout change; routine hourly compaction only touches new small files.

Airflow task	Duration (s)
apply_layout_and_compact	43.9
rewrite_deletes_and_manifests	22.8
expire_snapshots	23.1
remove_orphan_files	67.7

5.5 Fault tolerance (observed)

During the first freshness measurement the Docker VM ran out of memory and the operating system killed the Spark driver. The container restarted automatically, resumed from the checkpointed Kafka offsets and re-applied the interrupted micro-batch; because the MERGE is idempotent, no change was lost or duplicated and the subsequent consistency checks passed. The incident is visible in that measurement: median freshness stayed at 10.8 s, but the probes caught in the restart took up to 78.6 s (40 visible, 0 timed out). Memory limits were then reduced so that the steady-state total leaves headroom in the 8 GB VM, and freshness was re-measured (Section 5.2).

5.6 Correctness

Row counts and full-row checksums of every table were identical between PostgreSQL and Iceberg before and after the optimisation (Table 5); this check also ran after the initial load, after the workload, after a full replay of the Kafka topics into an empty lake, and in the integration test.

Table	Rows	Checksum match (before / after optimisation)
customer	149,932	True / True
nation	25	True / True
orders	1,513,192	True / True
region	5	True / True

Table 5 · PostgreSQL vs Iceberg (Trino `checksum()` over all source columns).

5.7 Functional demonstrations

- **Schema evolution:** `ALTER TABLE ... ADD COLUMN` and an integer→bigint change in PostgreSQL reached Iceberg within about 10 s as metadata-only changes; old rows read `NULL` for the new column.
- **Time travel:** the same order queried as of the current snapshot, a snapshot id and a timestamp returned the new and the old values; the CDC columns allow auditing which rows changed since a snapshot.
- **GDPR erasure:** a customer deleted in PostgreSQL disappeared from current queries within about 10 s (as position deletes) but was still readable through time travel; the purge DAG rewrote the affected files and expired history, after which the old snapshot no longer existed.

06 Technical challenges and lessons learned

Problem observed	Root cause	Resolution
MERGE commits failed with HTTP 500 under concurrent writes	REST catalog's SQLite store returned <code>SQLITE_BUSY</code>	Catalog metadata moved to a PostgreSQL JDBC store; lake rebuilt by replaying Kafka
MERGE failed: "Cannot add fieldId ... as an identifier field"	Iceberg 1.9 issue with identifier fields in Spark MERGE	Primary key kept as a table property
<code>remove_orphan_files</code> : no FileSystem for scheme "s3"	Procedure lists storage via Hadoop; stack uses S3FileIO	S3 listing supplied via <code>file_list_view</code> ; 24 h safety window kept
Join queries hit Trino's 432 MB per-node limit; later Trino ran at its container limit	Default sizing: heap = 80 % of the container, query memory = 30 % of heap	Explicit 1 GB heap, 600 MB query memory, spill-to-disk
Sorted rewrite of 1.5 M rows ran out of heap, then was SIGKILLED	1 GB driver too small; then uncapped off-heap memory exceeded the container	1.2 GB heap with capped direct/metaspaces memory; container limit sized for Airflow + job
High JSON parsing cost per event	Debezium JSON carries its schema (~3 KB)	zstd in Kafka; schema read from a string prefix instead of full parsing
Kafka Connect ran out of heap during the 1.5 M-row load	Debezium queue bounded by record count only	Queue bounded by bytes (128 MB); heap raised to 896 MB
Spark driver killed by the VM out-of-memory killer	Sum of container memory use exceeded the 8 GB VM	Driver and Trino memory trimmed; restart recovered with no data loss
Fitting 13 services into 8 GB	Laptop deployment	Per-container memory limits, local-mode Spark, shared base image

The main lesson is that the hard parts of CDC ingestion are operational: concurrency between writers, metadata growth, small files and observability matter as much as the happy-path data flow, and each is easier to get right with an explicit correctness check that can be re-run at any time.

07 Conclusion and future work

The project delivers a working, reproducible CDC lakehouse: changes made in PostgreSQL become queryable in Iceberg within seconds, the copy is provably identical to the source, schemas evolve without intervention, history is queryable and can be erased on request, and scheduled optimisation turns the streaming-induced small files into a layout that makes analytical queries 3.25× faster while cutting the number of stored objects by 89 %. Limitations are the single-node deployment, a 10 s minimum freshness from the micro-batch trigger, and that dropped source columns remain in Iceberg. Natural extensions are a comparison with Apache Hudi on the same workload, column-level lineage with Apache Atlas or OpenLineage, Avro with a schema registry to cut serialisation cost, deployment on Kubernetes (AKS) with Azure Data Lake Storage and Event Hubs, and Flink for lower-latency ingestion.

References

1. Apache Iceberg. *Table Spec, Spark procedures and maintenance documentation*. <https://iceberg.apache.org/docs/latest/>
2. Debezium. *Debezium connector for PostgreSQL*, version 2.7. <https://debezium.io/documentation/>
3. Apache Spark. *Structured Streaming Programming Guide*, version 3.5. <https://spark.apache.org/docs/3.5.6/>
4. Trino. *Iceberg connector*. <https://trino.io/docs/current/connector/iceberg.html>
5. Apache Kafka. *Documentation*, version 3.9. <https://kafka.apache.org/documentation/>
6. PostgreSQL Global Development Group. *Logical Replication*, PostgreSQL 16 documentation.
7. Apache Airflow. *Documentation*, version 2.10. <https://airflow.apache.org/docs/>
8. M. Armbrust, A. Ghodsi, R. Xin, M. Zaharia. *Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics*. CIDR 2021.
9. Transaction Processing Performance Council. *TPC Benchmark H Standard Specification*.

Appendix A – Reproducing the results

```
git clone <repository> floe && cd floe
./scripts/setup.sh          # prerequisites, .env, verified jar downloads, images
./scripts/run_demo.sh --full # stack, TPC-H sf1, workload, validation, demos, benchmark
# or step by step
make up                    # start all services
make load                  # TPC-H sf1 -> PostgreSQL; prints end-to-end ingestion throughput
make bench                 # freshness probes under load, then before/after optimisation benchmark
make validate              # PostgreSQL vs Iceberg row counts + checksums
python3 docs/report/build_report.py # regenerate this report from results/
```

Appendix B – Key configuration

```
-- Debezium connector (config/debezium/postgres-cdc.json, excerpt)
plugin.name=pgoutput    publication.name=dbz_publication    slot.name=debezium_shop
topic.prefix=pg         schema.include.list=shop           snapshot.mode=initial
decimal.handling.mode=string    column.propagate.source.type=shop\.*    heartbeat.interval.ms=10000

-- Iceberg table properties set by the streaming job
format-version=2    write.{delete,update,merge}.mode=merge-on-read    write.parquet.compression-codec=zstd

-- Spark catalog (config/spark/spark-defaults.conf)
spark.sql.catalog.lakehouse=org.apache.iceberg.spark.SparkCatalog    type=rest    uri=http://iceberg-
rest:8181
io-impl=org.apache.iceberg.aws.s3.S3FileIO    s3.endpoint=http://minio:9000    s3.path-style-access=true

-- Layout optimisation (src/topologies/spark/maintenance.py)
ALTER TABLE lakehouse.shop.orders ADD PARTITION FIELD months(o_orderdate);
ALTER TABLE lakehouse.shop.orders WRITE ORDERED BY o_orderdate, o_custkey;
CALL lakehouse.system.rewrite_data_files(table => 'shop.orders', strategy => 'sort',
    options => map('rewrite-all','true','delete-file-threshold','2','target-file-size-
bytes','134217728',
    'partial-progress.enabled','true'));

```